

System Overview Document: [Duets.ai]

Project Name:	Duets.ai
Version:	[1.1]
Date:	[Dec 10th, 2025]
Author(s):	[Deniz Gould, Junge Zhang, Arif Syraj]
Status:	[Final]
Sponsor/Stakeholder:	[Valerio Mirarchi]

Table of Contents

System Overview Document: Duets.AI	1
Document Revision History	3
1. Overview and Results	4
1.1. Project Summary	4
1.2. Key Accomplishments	5
1.3. Lessons Learned	6
1.4. Next Steps & Future Enhancements	6
2. Software Architecture	8
2.1. Technology Stack	8
2.2. Architectural Overview & Diagram	8
3. Design	11
3.1. Major Software Modules	11
3.2. Database Schema	14
3.3. Information Flow	19
3.4. Use Cases and User Interface Flow	20
4. Build Instructions	25
4.1. Prerequisites	25
4.2. Installation and Build Steps	25
4.3. Testing	27
4.3.1. Software Correctness (Automated Tests)	28
4.3.2. Usability Testing	29

Document Revision History

Version	Date	Author	Description of Changes
1.0	[Dec 9]	[Deniz Gould]	Initial draft of the System Overview Document.
1.1	[Dec 10]	[Arif Syraj]	Additions to sections 1.2, 3.1, 3.2, 3.3, 3.4, 4

1. Overview and Results

This section provides a high-level summary of the project's achievements, learnings, and potential future direction.

1.1. Project Summary

Duets.ai is an AI-augmented language learning platform that enhances live human-led lessons with automated feedback, real-time analytics, and personalized practice tools. The Fall 2025 project extended the platform in three major areas:

1. a fully functional scoring and tracking system,
2. infinite exercise generation, and
3. an AI-assisted peer-to-peer chat experience.

These additions strengthen the platform's core value proposition — enabling students to reinforce learning outside class while giving teachers and institutions clear insight into performance trends. Students now receive targeted exercises, view their progress visually, and practice freely with AI support, while teachers gain dashboards summarizing class mastery, session frequency, and cohort performance. Institutions can access aggregated reporting through the updated admin dashboard.

1.2. Key Accomplishments

Deliverable 1 — Scoring & Tracking System (Students, Teachers, Institutions)

- Implemented complete session-level and all-time scoring.
- Added metrics including streaks, mastery indicators, accuracy trends, retries per question, and best exercise type.
- Developed **student dashboard**
- Developed **teacher dashboard**
- Created backend endpoints: api/student-score, api/teacher-metrics
- Integrated institution-level list and teacher metrics display.

Deliverable 2 — Infinite and Session Exercise Generation

- End-to-end pipeline for generating unlimited exercises as well as exercises tied to a meeting.
- LLM-driven generation with categories (translation, fill-in-the-blank, and true/false).

- Created new Bubble.io pages and fixed buggy ones: exercise generation screen, answering interface, reveal-answer flow, feedback flow, and response history.
- Created backend endpoints: api/transcript-details, api/generate-exercise, api/exercise-count, api/save-responses-sli api/student-responses-retrieval.
- Migration of ~10,000 historical exercises from Azure Blob to Supabase.
- Updated prompts to greatly improve quality of returned exercises, including past exercises and the meeting transcript.

Deliverable 3 – AI P2P Chat System

- Fully functional chat interface between students.
- AI-moderated suggestions and message guidance (LLM hint pipeline).
- Contact list, message history retrieval, and message posting endpoints.
- Email notification for unread messages.
- Endpoints: api/aip2p-chat-list-contact-and-message, api/aip2p-chat-add-message, api/aip2p-chat-llm-help.

Deliverable 4 – Backend Infrastructure Improvements

- Major code cleanup and refactoring.
- New tables added to Supabase; schema finalized for exercises and student responses.
- Deprecated code removed and flow unified across endpoints.
- Backend reliability improvements for SLI generation pipeline.
- Convert pipeline to use Supabase for structured data on top of Azure.
- Updated to using gpt-4.1, leading to better and faster functionality at a cheaper rate by 4-8 times.

Deliverable 5 – Frontend Redesign & Navigation Updates (Bubble.io)

- New navigation bar for students, teachers, and institutions.
- Redesigned dashboards (student, teacher).
- Added infinite exercises and chat tabs.

Key Metric Achieved

- The system now supports **complete end-to-end progress visibility** for students and teachers.
- Infinite exercise generation supports **arbitrary volume with stable generation latency (<10s)**.
- AI P2P chat round-trip latency remains within acceptable bounds for Bubble-based asynchronous updates.

1.3. Lessons Learned

Technical Lessons

- **Bubble.io constraints:** Limited version control and lack of staging environments slowed frontend iteration.
- **Supabase PostgreSQL** proved far superior to Azure Blob for structured data – migration validated this decision.
- **Django endpoint consolidation** reduced API duplication and clarified flows (exercise generation, scoring, P2P chat).
- **LLM prompt engineering** significantly improved reliability of SLI and exercise generation.

Process Lessons

- Agile workflow with GitHub Issues + Notion improved team organization.
- Early weekly sponsor meetings clarified requirements and avoided rework.
- Parallel backend/frontend development was challenging due to Bubble's limited tooling.

Domain Lessons

- Teachers value **aggregated cohort analytics** more than per-student summaries.
- Students are highly motivated by **streaks, visual progress, and infinite practice**.
- AI suggestions in chat need guardrails; students rely heavily on feedback tone and clarity.

1.4. Next Steps & Future Enhancements

Short-Term (Next 3 months)

- Expand teacher dashboards with per-topic mastery heatmaps.
- Add institution-level trend reports (weekly/monthly engagement).
- Improve AI P2P moderation with conversation-topic constraints.

Medium-Term (6–12 months)

- Voice-based AI P2P practice.
- Gamification features (badges, rewards).
- Student-level longitudinal mastery predictions using lightweight ML models.

Long-Term (Beyond 12 months)

- Mobile application for students.

- White-label version for universities and private language schools.
- Integration with LMS systems such as Canvas or Moodle.

2. Software Architecture

This section describes the high-level structure of the system, the technology choices, and how the components interact

2.1. Technology Stack

Layer	Technology
Frontend	<i>Bubble.io (no-code UI builder), custom API integrations</i>
Backend (API)	<i>Python, Django, LangChain, Azure Python SDK</i>
Database	<i>Supabase (PostgreSQL) – structured data; Azure Blob Storage – transcripts, SLIs</i>
Cloud/Infrastructure	<i>Azure App Services, Azure Container Registry, Docker</i>
DevOps & Tools	<i>GitHub Actions, Postman, Docker Compose, Notion, Trello</i>

2.2. Architectural Overview & Diagram

The system follows a multi-service architecture:

Frontend (Bubble.io)

- Handles routing, UI layout, user actions, and calls backend endpoints.

Backend-Django

- Core logic layer: scoring, exercise generation, AI prompts, chat system, SLI generation.
- Exposes REST endpoints consumed by Bubble.io.

Transcription Service

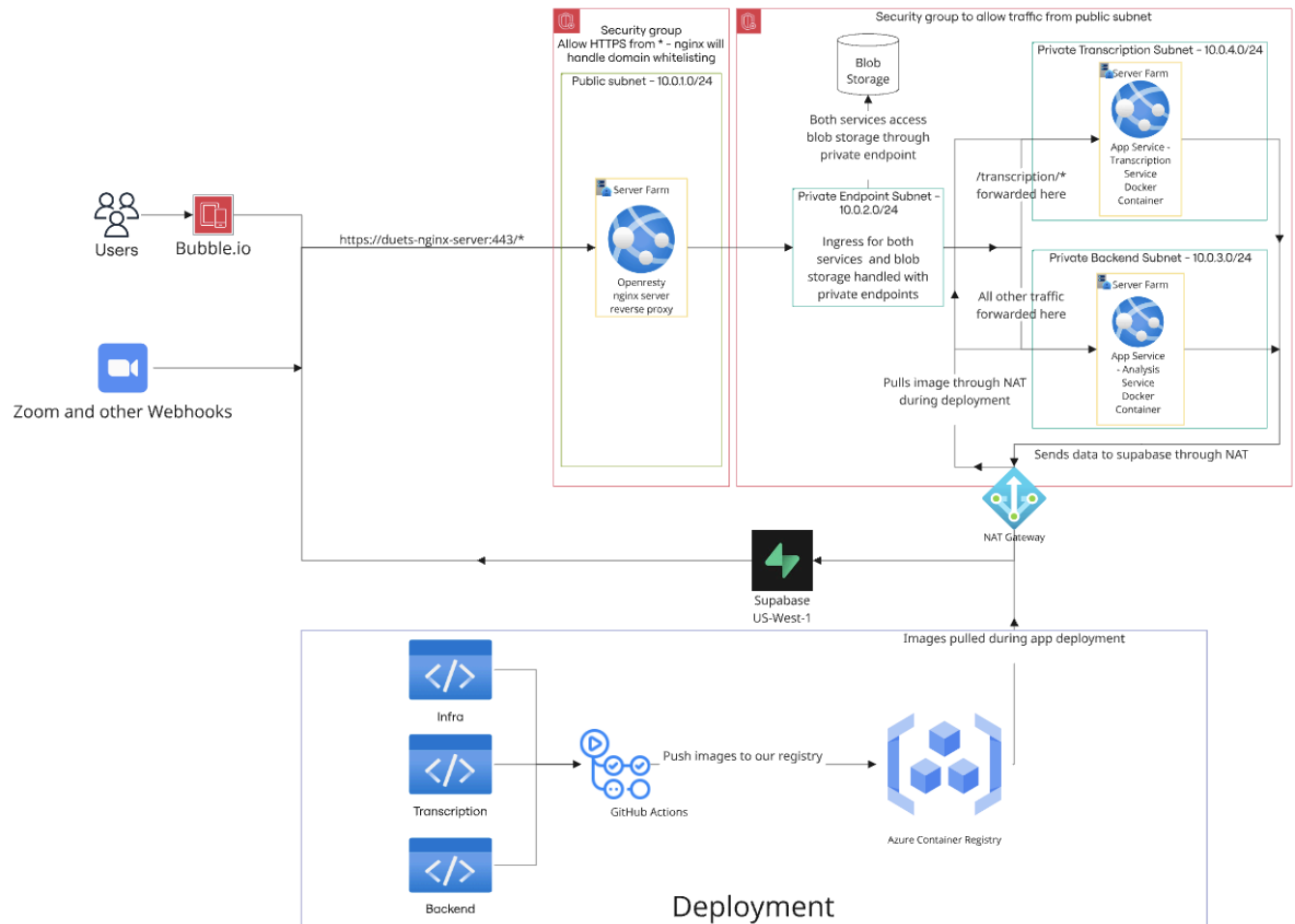
- Listens to Zoom webhook. Downloads and extracts meeting session recordings when they become available. Then starts the transcribe and summarize pipeline, which outputs summaries called SLI, Smart Learning Insights.
- Uploads processed data to Azure Blob and indexes them on Supabase.

Database Layer

- Supabase stores all structured data (students, exercises, responses, metrics).
- Azure Blob stores meeting recordings, transcripts and SLI summaries.

LLM Layer

- OpenAI (via LangChain) generates exercises, SLIs, and chat suggestions.



[Figure 1: High-level system architecture showing the frontend, API gateway, backend services, and shared database.]

3. Design

This section dives into the detailed design of the system's major components and workflows.

3.1. Major Software Modules

The backend is organized around clean, well-defined API modules that each handle a specific vertical of the system: meeting data, exercise generation, exercise retrieval, scoring, saving responses, teacher metrics, and AI chat.

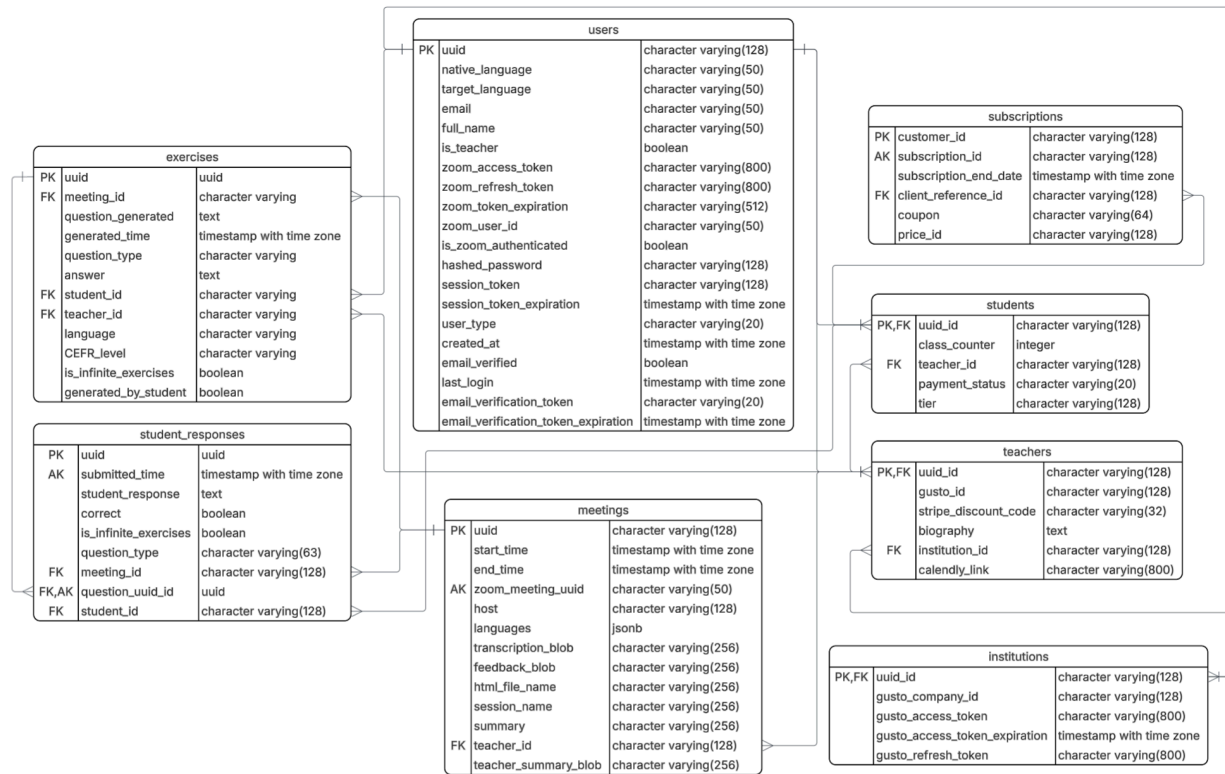
- Module: Meeting & Transcript Retrieval
 - Purpose: Provides access to lesson history, transcripts, SLI summaries, and teacher feedback by retrieving meeting metadata and associated content stored in Azure Blob and Supabase.
 - Key Functions: `UserMeetingsView.get()`, `UserMeetingDetailsView.get()`, `OutputContentView.get()`, `HTMLContentView.get()`
- Module: Feedback Generation Pipeline
 - Purpose: Generates structured session feedback and summaries from transcripts via LLM and stores them as blobs and DB references.
 - Key Functions: `load_config()`, `extract_prompts_and_formats()`, `query_llm_and_parse_response()`, `serialize_responses()`, `save_output_blob_to_db()`, `save_summary_to_db()`
- Module: Transcript Processing & Azure Blob Utilities
 - Purpose: Processes raw meeting transcripts and manages reading and writing transcript and feedback files in Azure Blob Storage.
 - Key Functions: `TranscriptProcess.process()`, `TranscriptProcess.chunk_transcript()`, `AzureUtilities.retrieve_file_from_blob()`, `AzureUtilities.create_transcript_string_from_blob()`, `AzureUtilities.upload_blob()`
- Module: Prompt & LLM Configuration
 - Purpose: Centralizes GPT model configuration, prompt templates, and structured output parsers shared by all AI-driven features.
 - Key Functions: `PromptsManager.get_prompt_zero()`, `PromptsManager.get_prompt_infos()`

PromptsManager.get_completed_exercise_prompts(),
PromptsManager.get_completed_categorized_exercise_prompts(),
ParsersManager.get_parsers(),
ParsersManager.get_categorized_exercise_parsers()

- Module: Supabase & Meeting Utilities
 - Purpose: Resolves student participants and meeting identifiers when linking meetings, blobs, and transcript-derived outputs.
 - Key Functions: get_participant_uuid_from_meeting(),
extract_meeting_id_from_blob_name()
- Module: Exercise Generation Pipeline
 - Purpose: Generates personalized session-based and infinite exercises using LLMs with transcript and history-based context.
 - Key Functions: ExerciseView.get(),
ExerciseGenerator.generate_new_exercises(),
PromptsManager.build_prompt(),
ExerciseGenerator._is_placeholder_exercise()
- Module: Exercise Retrieval Service
 - Purpose: Retrieves exercises for meetings, infinite mode, or full student history, and returns exercise counts.
 - Key Functions: TranscriptDetailsView.get(), ExerciseCountView.get(),
OutputContentView.get()
- Module: Student Response Handling
 - Purpose: Saves and normalizes student responses for scoring and analytics in both session and infinite modes.
 - Key Functions: SaveResponsesSLI.post(),
StudentResponsesRetrieval.get()
- Module: Scoring & Tracking Engine
 - Purpose: Computes student performance metrics including accuracy, retries, lesson scores, streaks, and best-performing exercise formats for dashboard display.
 - Key Functions: StudentScoreView.get(), get_score_summary(),
get_weekly_streak_summary(), get_best_question_type(),
resolve_latest_meeting_with_responses()

- Module: Teacher Metrics Engine
 - Purpose: Aggregates session activity and student performance to provide instructors with insights such as session frequency, score distributions, and quartile metrics.
 - Key Functions: `TeacherMetricsView.get()`, `_calculate_average_sessions_per_week()`, `_calculate_student_score_stats`
- Module: AI P2P Chat Service
 - Purpose: Enables peer-to-peer student chatting with AI-assisted grammar and phrasing suggestions, while retrieving contacts and conversation histories. Email notifications to users when they have a message left unread for some time.
 - Key Functions: `Aip2pChatListContactAndMessageView.get()`, `Aip2pChatAddMessage.post()`, `Aip2pChatLlmHelpView.post()`

3.2. Database Schema



[Figure 2: Comprehensive schema for database]

- Table: `users` - Stores basic user account information for auth and access to zoom, and also basic personal information.

users		
PK	uuid	character varying(128)
	native_language	character varying(50)
	target_language	character varying(50)
	email	character varying(50)
	full_name	character varying(50)
	is_teacher	boolean
	zoom_access_token	character varying(800)
	zoom_refresh_token	character varying(800)
	zoom_token_expiration	character varying(512)
	zoom_user_id	character varying(50)
	is_zoom_authenticated	boolean
	hashed_password	character varying(128)
	session_token	character varying(128)
	session_token_expiration	timestamp with time zone
	user_type	character varying(20)
	created_at	timestamp with time zone
	email_verified	boolean
	last_login	timestamp with time zone
	email_verification_token	character varying(20)
	email_verification_token_expiration	timestamp with time zone

- Table: `students` - Information for students for tracking current subscription.

students		
PK,FK	uuid_id	character varying(128)
	class_counter	integer
FK	teacher_id	character varying(128)
	payment_status	character varying(20)
	tier	character varying(128)

- Table: subscriptions - Information for each single subscription.

subscriptions		
PK	customer_id	character varying(128)
AK	subscription_id	character varying(128)
	subscription_end_date	timestamp with time zone
FK	client_reference_id	character varying(128)
	coupon	character varying(64)
	price_id	character varying(128)

- Table: institutions - Information for each institution.

institutions		
PK,FK	uuid_id	character varying(128)
	gusto_company_id	character varying(128)
	gusto_access_token	character varying(800)
	gusto_access_token_expiration	timestamp with time zone
	gusto_refresh_token	character varying(800)

- Table: `teachers` - Information for teachers for payment and calendar.

teachers		
PK,FK	uuid_id	character varying(128)
	gusto_id	character varying(128)
	stripe_discount_code	character varying(32)
	biography	text
FK	institution_id	character varying(128)
	calendly_link	character varying(800)

- Table: `meetings` - Information for each tutoring meeting session on Zoom, most importantly the reference to corresponding blob objects in Azure storage for transcription and summary.

meetings		
PK	uuid	character varying(128)
	start_time	timestamp with time zone
	end_time	timestamp with time zone
	zoom_meeting_uuid	character varying(50)
AK	host	character varying(128)
	languages	jsonb
	transcription_blob	character varying(256)
	feedback_blob	character varying(256)
	html_file_name	character varying(256)
	session_name	character varying(256)
	summary	character varying(256)
	teacher_id	character varying(128)
FK	teacher_summary_blob	character varying(256)

- Table: `exercise` - Information for each generated exercise, the question, expected answer, owner student and based on which meeting it is generated.

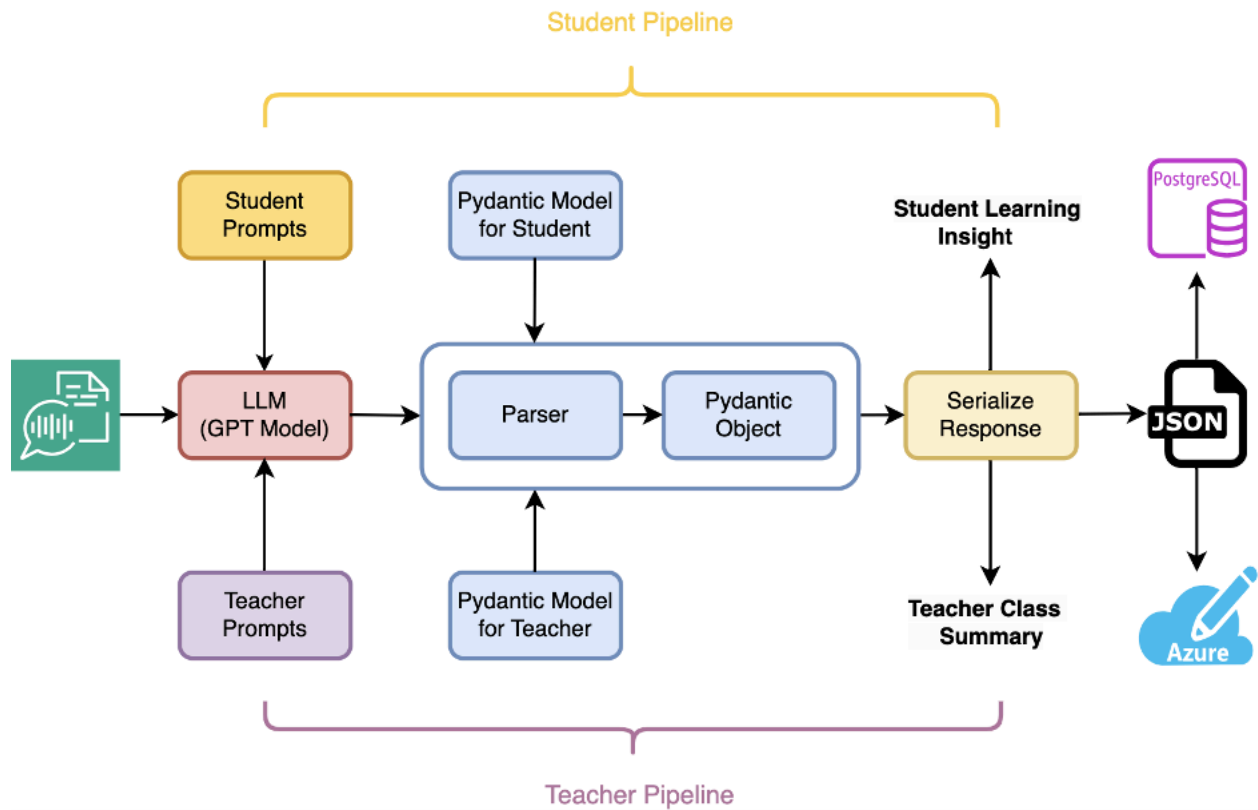
exercises		
PK	uuid	uuid
FK	meeting_id	character varying
	question_generated	text
	generated_time	timestamp with time zone
	question_type	character varying
	answer	text
FK	student_id	character varying
FK	teacher_id	character varying
	language	character varying
	CEFR_level	character varying
	is_infinite_exercises	boolean
	generated_by_student	boolean

- Table: `student_responses` - Information for each submission of a question, the response content, time of submission and result.

student_responses		
PK	uuid	uuid
AK	submitted_time	timestamp with time zone
	student_response	text
	correct	boolean
	is_infinite_exercises	boolean
	question_type	character varying(63)
FK	meeting_id	character varying(128)
FK,AK	question_uuid_id	uuid
FK	student_id	character varying(128)

3.3. Information Flow

Example: AI Pipeline Flow



[Figure 3: Flow diagram for AI services after a meeting's conclusion]

The process can be summarized below:

1. **Input Stage:** Preprocessed meeting transcripts and prompts are used as input data to be ingested into the pipeline.
2. **Load GPT Model:** GPT-4.1 by OpenAI is used to analyze the input data and generate meaningful responses for student learning insights, student exercise and teacher class summary.
3. **Querying the Model:** The loaded GPT model is queried with specific prompts using Langchain, which includes the meeting transcript.
4. **Parsing and Structuring the Response:** The response from the GPT model is parsed using a parser. A Pydantic model is employed to define the desired structure or schema of the output. This ensures that the response conforms to a specified format, making it reliable for downstream processes.
5. **Generating and Storing the Output:** The structured data from the Pydantic object is serialized into a JSON file. This JSON file is then uploaded to Azure Blob

Storage for secure and scalable storage. A reference to the stored file - its blob name, a unique identifier - is saved in a database for easy retrieval and management.

Example: "Flow for Submitting and Processing New Feedback"

1. User submits feedback via the Web Client.
2. Web Client sends a `POST /api/feedback` request to the API Gateway.
3. API Gateway routes the request to the Data Ingestion Service.
4. Data Ingestion Service validates the data and stores the raw entry in the `raw_feedback` table.
5. Data Ingestion Service emits a `feedback.received` event.
6. The Analytics Engine listens for this event, fetches the new raw feedback, and processes it.
7. The Analytics Engine saves the resulting insight into the `processed_insights` table.
8. The Web Client can now fetch the updated insights via a `GET /api/insights` request.

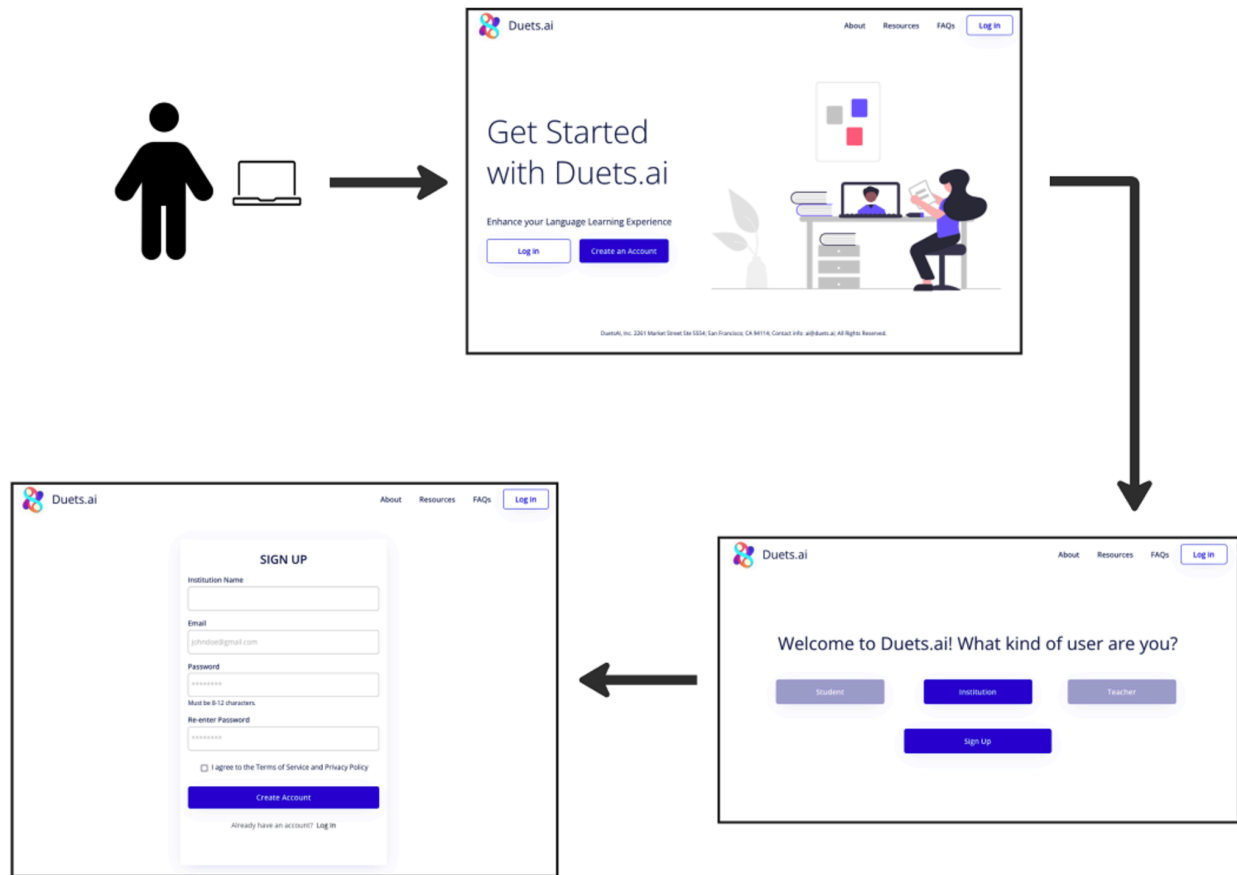
3.4. Use Cases and User Interface Flow

Primary Use Cases:

1. UC-1: Institution Registration Flow
2. UC-2: Teacher Registration Flow
3. UC-3: Student Registration Flow
4. UC-4: Typical End-to-End Use Case Flow

UC-1: Institution Registration Flow

The institution user is the user at the highest level of hierarchy on the platform. They have access to not only view the teachers and students that are associated with the specific institution, but also the information viewable from these other types of users. Institution users follow a simple registration flow:



[Figure 4: Registration flow for an institution]

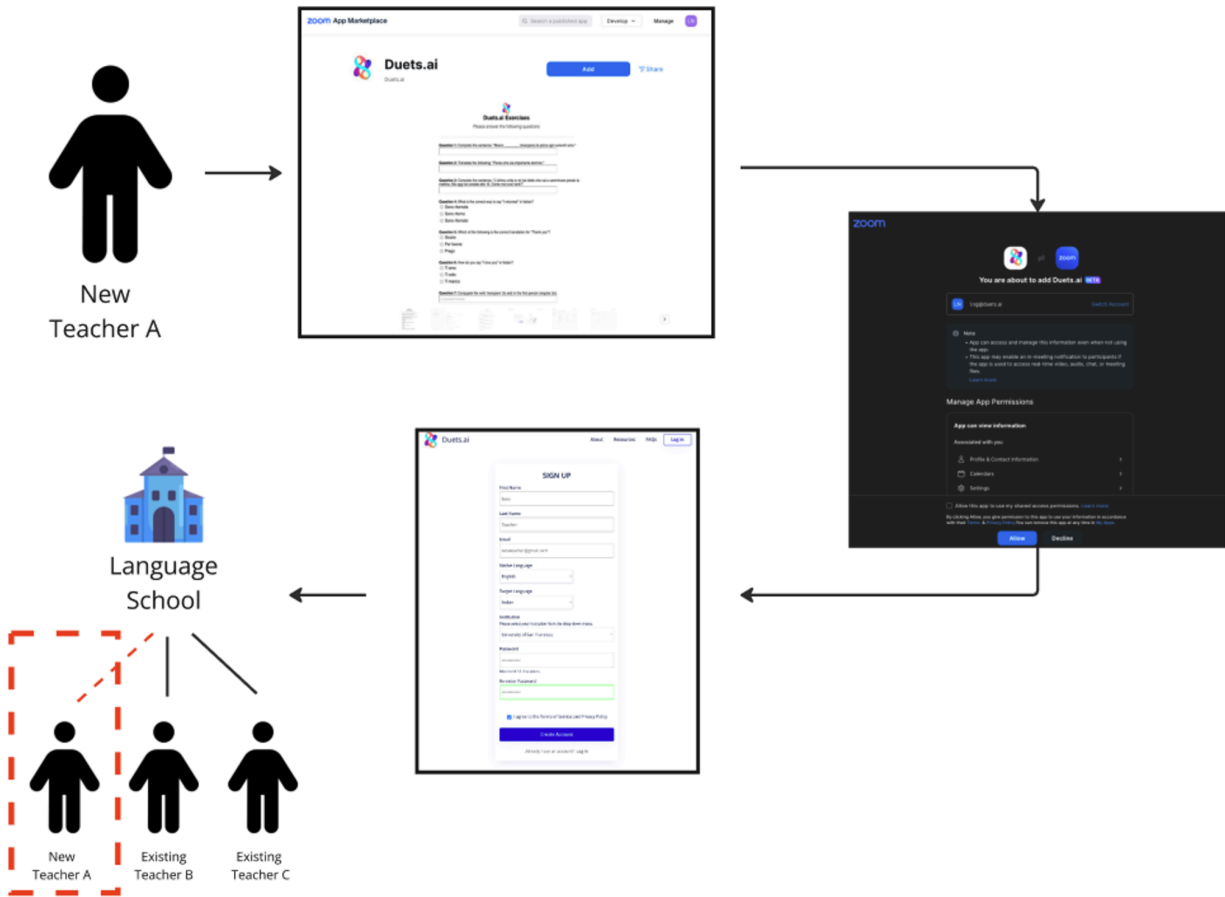
Upon successful registration, users are asked to verify their email by entering a confirmation code before gaining full access to the platform.

UC-2: Teacher Registration Flow

The teacher user is the next level of hierarchy down. Teachers must be associated with an institution in order to sign up, and teachers have 2 methods of initiating the registration process:

1. Teachers can proceed to the Duets.ai platform and follow the regular registration process.
2. Teachers can go directly to the Duets.ai app on the Zoom App Marketplace and download the application to initiate the registration process.

The below diagram shows the flow for method 2:



[Figure 5: Registration flow for a teacher]

In this registration flow, the teacher grants permission to the Duets.ai platform to gain access to the teacher's Zoom account information. When the teacher is redirected to the Duets.ai registration page, the Zoom authorization code is sent as a URL parameter, which is then passed to the Duets.ai backend when the registration form is submitted. This initiates the OAuth flow between Duets.ai and Zoom, whereupon Duets.ai receives the Zoom Access Token and Zoom Refresh Token for this specific user. This information can be used in the future to make API requests to the Zoom platform on behalf of the user, and Duets.ai also received webhook notifications for this user's actions on the Zoom platform.

If the teacher follows registration flow 1 by signing up directly via Duets.ai website, they are prompted to redirect to the Zoom App Marketplace page to download the Zoom application before being redirected back to the dashboard page for completion of the OAuth process before gaining access to the Duets.ai platform.

During the registration process, the Duets.ai backend also reaches out to the Stripe platform to generate a unique discount code for the teacher. This discount code is a unique identifier for the

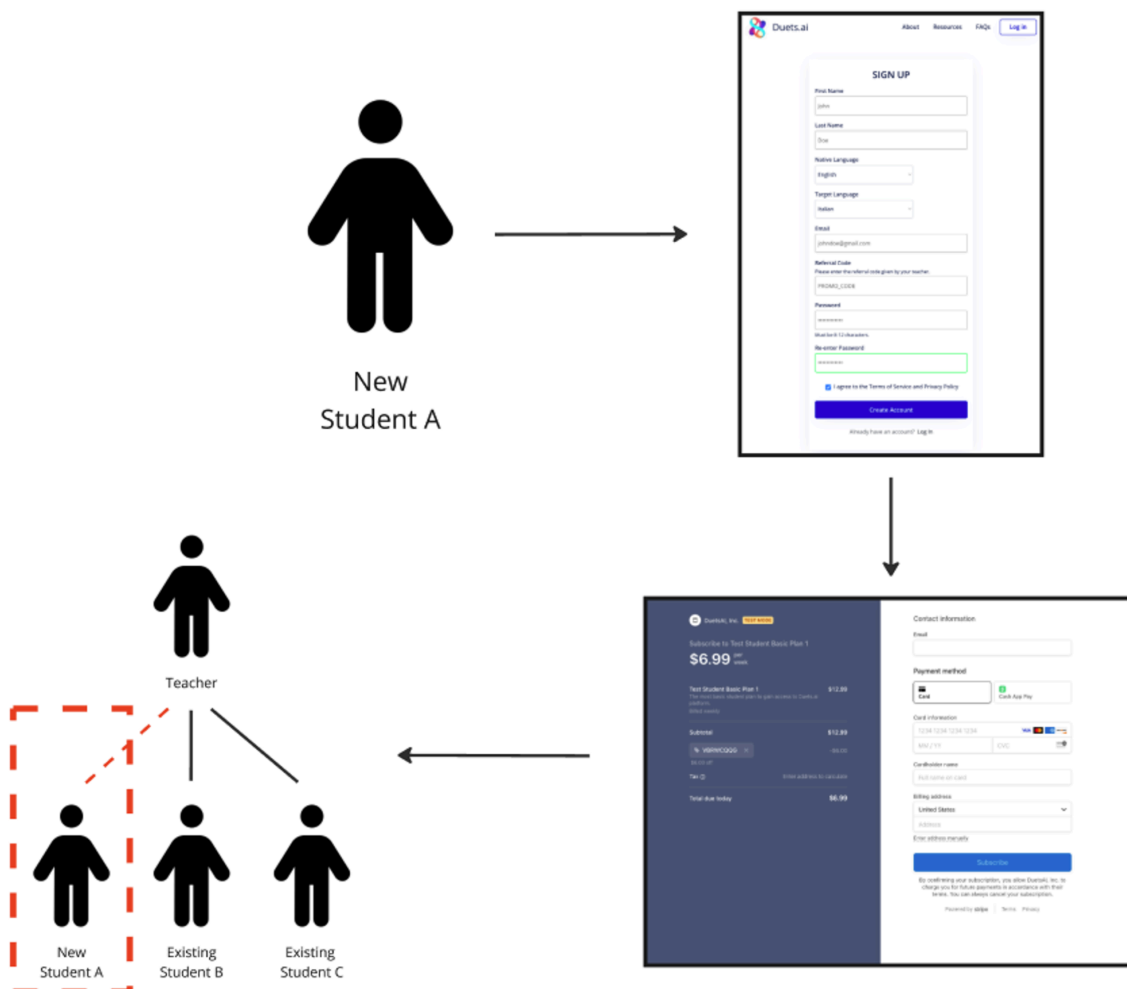
teacher, and allows students to sign up for the platform using this unique code and be associated with an existing teacher on the platform.

Teacher User Configuration

Upon successful registration, teachers can add a calendly link to their account so that students can schedule a lesson with the teacher via the Duets.ai platform directly. Other user configurations such as updating profile information is also available, and other features/functionalities can be added in the future.

UC-3: Student Registration Flow

The student user is the user at the lowest hierarchical level. The registration flow for the student is described below:



[Figure 6: Registration flow for a student]

In this registration flow, a student goes to Duets.ai website and registers as a student. The student must provide a valid discount code so that the user is associated with a

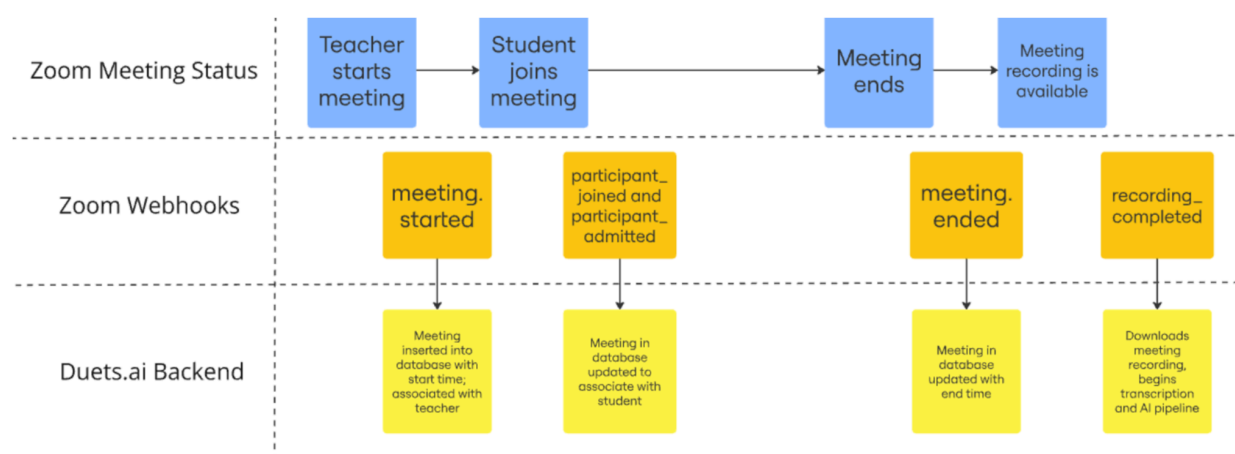
teacher upon signup. Upon successful creation of the user on the platform, the student is redirected to a Stripe-hosted payment page where the user has to complete a subscription payment before gaining full access to the platform.

When a Stripe payment is completed, Stripe sends a webhook to the Duets.ai backend which enables us to update the user's payment status and enable full access. Minimal payment reference information is stored in the database so that future functionalities such as cancellation or upgrading of subscription plans can be implemented. A Stripe-hosted customer portal can also be linked for students to manage their own subscription. There is also a currently not-in-use but implemented logic of requiring the user to install the Zoom app prior to full access. This would require the user to be redirected to the Zoom App Marketplace page and be redirected back, similar to the teacher flow

UC-4: Typical End-to-End Use Case Flow

Once users are registered on the platform, students and teachers can go through the below typical end-to-end use case flow:

1. Student schedules meeting with teacher via Calendly link embedded on Duets.ai website
2. Teacher accepts meeting and sends Zoom link to student for use (outside of platform)
3. Meeting starts, webhooks ingestion begin. The below diagram shows the stages of the Zoom meeting and the webhooks our platform receives and the actions completed on our side.



[Figure 7: Sequential operations across Zoom, its webhooks, and the backend during and after a meeting]

4. Meeting ends, transcription starts in the Transcription Service module
5. Once transcription is completed, the database is updated with the transcription file name for the particular meeting. A Supabase webhook is then sent to the AI Service module to start the AI-generated work.
6. Upon completion of AI-generated content, the student will receive an email summary of the session, and the session's information and exercises are available on the website for the teacher and student to see.

4. Build Instructions

This section provides all necessary instructions for a new developer (or the sponsor) to build, run, and test the system from scratch.

4.1. Prerequisites

- docker 28.x.x or higher

This project uses docker to ship the programs. Requirements are automatically installed in virtualized environments.

4.2. Installation and Build Steps

The project is already up and running on duets.ai.

To run the project in a local development environment for testing purposes, follow these steps.

1. Clone the Repository

The GitHub repositories contain submodules. Some of these references to submodules use ssh urls. To automatically checkout submodules it is required to setup ssh config and credentials for GitHub.

```
git clone --recursive https://github.com/duets-ai/backend-django.git
git clone --recursive https://github.com/duets-ai/transcription-service.git
```

2. Populate .env files for containers

The .env file is not included in the repository for security reasons. Content of the .env files are kept in Trello.

They should be placed at `repo/.env` where repo is the one you are building (backend, transcription, or nginx)

3. Run the Application

```
docker compose up at backend-django
```

The application will be accessible at <http://localhost:8000>

To run both the backend and transcription services at once:

First, a docker network has to be created so these containers can find each other.

Secondly, the port numbers for both services have to be different so there is no conflict.

Since there is no code for a frontend, these services can only be interacted with through other scripts or through the CLI. The containers can be reached by sending requests to

```
localhost:<port>/api/<endpoint>
```

Change the port:

In either the transcription or backend-django repo, change the port number to a different number. Do this by going to docker-compose.yml; the port option can be found underneath web/services. For example:

```
web:
  services:
    ports:
      - "8000:8000"
```

To

```
web:
  services:
    ports:
      - "8001:8000"
```

Create the network:

```
docker network create duets-network
```

Ensure the transcription service and backend service are listening on different ports by changing this parameter in their respective docker-compose.yml files

```
# Terminal 1 - Start backend-django
cd backend-django
docker compose up
# Terminal 2 - Start transcription-service
cd transcription
docker compose up
```

4.3. Testing

4.3.1. Software Correctness (Automated Tests)

There are 3 types of testing done on this application:

Automated Test Cases for Registration

Test cases are written for our backend using Django's unit test library. The main tests are written to ensure that the registration flows are able to properly create the different types of users. These can be found within the backend-django repository under `userauth/tests.py`.

Instructions on how to write and run tests are available from Django's website:

<https://docs.djangoproject.com/en/5.1/topics/testing/overview/>

Manual Testing

Manual testing is also done on other parts of the projects, including parts of the AI pipeline and the frontend application.

For the frontend application, endpoints for the backend need to be defined by orchestrating a successful backend API call with a correct response. This implicitly required testing, as the parameters that can be used in the frontend from the API call depend on the response from the backend for that call. Every time new features were deployed, the endpoint would have to be set up by sending a request to the hosted backend.

For the AI pipeline, refinement of AI prompts were done manually using OpenAI's API calls. Depending on the quality and format of the output, prompts were changed and refined.

For the prompts related to exercises, once a successful prompt has been found, several hundred exercises are generated. Basic, necessary semantic tests such as confirming every question has instructions attached to it, and that the number of questions are as expected, are then performed automatically. The test script for this is available in the backend-django repo as `test_exercise_generation.py`. Instructions on how to run it are included in the README of the `dev` branch:

```
python test_exercise_generation.py --user_id <student_uuid>
[--uuid <meeting_uuid> | --infinite-exercises]
```

4.3.2. Usability Testing

Beta Tester Feedback

The AI-generated content and overall usability of the web application and its functionalities are evaluated by the student users, students working on this project, and the sponsors. As the generated content requires domain specific knowledge, the feedback from beta testers are important so that we can understand if the content is reasonable, accurate, and useful. This feedback guides our priorities in feature development.